TP 5 — Sécurisation d'une API REST avec Spring Boot

Ralph Bou Nader

Master — Ingénierie Informatique

Objectifs du TP

Ce TP vous fait implémenter, sur une API REST concrète, les mécanismes de sécurité vus au Chapitre 7 :

- Authentification par clé API et par JWT.
- Gestion des rôles et autorisation (RBAC).
- Protection contre les attaques courantes (Injection, XSS).
- Limitation de débit (rate limiting, anti-abus).
- Gestion centralisée des erreurs et bonnes pratiques HTTPS/CORS.

L'API gérera une liste d'utilisateurs fictifs, avec un rôle associé (`user` / `admin`).

Prérequis

- Java JDK 17 ou plus récent.
- Spring Boot 3.x (généré via Spring Initializr).
- Eclipse.
- Postman.

Remarque : ce TP utilise Spring Boot 3.x, dont les imports commencent par `jakarta.*` (et non `javax.*` comme dans les versions antérieures à 2023).

1. Préparation du projet

1. Créez un projet Spring Boot (`tp-securite-api`) sur Eclipse avec les dépendances suivantes :
   - `spring-boot-starter-web`
   - `spring-boot-starter-security`
   - `spring-boot-starter-validation`
   - `io.jsonwebtoken:jjwt-api:0.12.5`
   - `io.jsonwebtoken:jjwt-impl:0.12.5` (scope `runtime`)
   - `io.jsonwebtoken:jjwt-jackson:0.12.5` (scope `runtime`)
   - `org.apache.commons:commons-text`
2. Créez le package racine `com.example.tp`, avec les sous-packages :
   - `model`
   - `controller`
   - `service`
   - `security`

But pédagogique : comprendre comment Spring gère la sécurité à différents niveaux (filtrage, authentification, autorisation).

2. Exercice 1 — Authentification par clé API

Objectif : mettre en place une première couche de sécurité basée sur une clé d'accès unique.

1. Créez une classe `ApiKeyFilter` dans le package `security`, qui étend `OncePerRequestFilter` (classe Spring recommandée pour garantir qu'un filtre ne s'exécute qu'une seule fois par requête).
2. Ce filtre doit intercepter chaque requête HTTP et vérifier la présence de l'en-tête :

```
Authorization: ApiKey 123456abcdef
```

3. Si la clé est absente ou incorrecte, renvoyez une erreur `401 Unauthorized`.
4. Sinon, laissez la requête continuer vers le contrôleur.

Indice technique (Spring Boot 3.x / Jakarta) :

```java
package com.example.tp.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;

@Component
public class ApiKeyFilter extends OncePerRequestFilter {

    private static final String CLE_VALIDE = "ApiKey 123456abcdef";

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws IOException, jakarta.servlet.Servle
        String key = request.getHeader("Authorization");
        if (key == null || !key.equals(CLE_VALIDE)) {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            return;
        }
        filterChain.doFilter(request, response);
    }
}
```

Questions à consigner :

1. Pourquoi cette méthode d'authentification est-elle considérée comme « faiblement sécurisée » ?
2. Comment pourrait-on renforcer ce mécanisme sans tout changer ?

3. Exercice 2 — Authentification par JWT et gestion des rôles

Objectif : générer et vérifier un token JWT signé pour chaque utilisateur connecté.

1. Créez une classe `JwtUtil` contenant :
   - une clé secrète, générée avec l'API moderne de JJWT (voir indice ci-dessous) ;
   - une méthode `generateToken(String username, String role)` ;
   - une méthode `validateToken(String token)`.

Indice de génération du token (API JJWT 0.12.x — la syntaxe a changé depuis l'ancienne version) :

```java
package com.example.tp.security;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import javax.crypto.SecretKey;
import java.util.Date;

public class JwtUtil {

    // Génère une clé secrète robuste (256 bits minimum pour HS256)
    private final SecretKey secretKey = Keys.hmacShaKeyFor(
        "mySecretKeyMySecretKeyMySecretKey123!".getBytes()
    );

    public String generateToken(String username, String role) {
        return Jwts.builder()
            .subject(username)
            .claim("role", role)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + 3600_000)) // 1h
            .signWith(secretKey)
            .compact();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser().verifyWith(secretKey).build().parseSignedClaims(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

    public String extraireRole(String token) {
        return Jwts.parser().verifyWith(secretKey).build()
            .parseSignedClaims(token).getPayload().get("role", String.class);
    }
}
```

Remarque : l'ancienne syntaxe `Jwts.builder().signWith(SignatureAlgorithm.HS256, SECRET)` (avec une clé en `String`) est dépréciée. La version actuelle exige une `SecretKey` typée, générée via `Keys.hmacShaKeyFor(...)`.

2. Créez un contrôleur `AuthController` :
   - Endpoint `/auth/login` qui prend un `username` et un `password`.
   - Vérifie des identifiants simulés (ex. `admin` / `admin123`).
   - Retourne un JWT contenant le rôle.
3. Créez un filtre `JwtFilter` (sur le modèle d'`ApiKeyFilter`, en `OncePerRequestFilter`) :
   - Récupère le token dans l'en-tête `Authorization: Bearer ...`.
   - Vérifie la validité et extrait le rôle.
   - Ajoute ce rôle dans le contexte de sécurité Spring (`SecurityContextHolder`).
4. Créez deux endpoints :
   - `/utilisateurs/user` accessible à tout utilisateur authentifié.
   - `/utilisateurs/admin` réservé aux utilisateurs avec le rôle `admin`.

Questions à consigner :

1. Quelle est la différence entre un token **signé** et un token **chiffré** ?
2. Pourquoi les JWT ne doivent-ils jamais contenir de données sensibles (mot de passe, numéro de carte…) ?

4. Exercice 3 — Protection contre les attaques XSS et injections

Objectif : filtrer et valider toutes les entrées utilisateur avant de les stocker.

1. Créez une entité `Utilisateur` avec les champs : `id`, `nom`, `email`, `description`.
2. Ajoutez des annotations de validation (rappel du TP2) sur le DTO de requête :

```
import jakarta.validation.constraints.*;

public class UtilisateurRequest {
    @NotBlank(message = "Le nom est obligatoire")
    private String nom;

    @Email(message = "Email invalide")
    private String email;

    private String description;
    // getters et setters
}
```

3. Dans `UtilisateurController`, avant de sauvegarder un utilisateur, nettoyez le texte libre pour éviter les attaques XSS :

```
import org.apache.commons.text.StringEscapeUtils;

u.setDescription(StringEscapeUtils.escapeHtml4(request.getDescription()));
```

4. Testez en envoyant une description contenant du code HTML ou JavaScript, par exemple `<script>alert('xss')</script>`.

Questions à consigner :

1. Quelle est la différence entre une injection SQL et une injection XSS ?
2. Pourquoi la validation côté client (JavaScript) ne suffit-elle pas à protéger l'API ?

5. Exercice 4 — Implémentation d'un Rate Limiting

Objectif : empêcher un utilisateur ou une IP d'abuser de l'API.

1. Créez un filtre `RateLimitFilter` dans `security` (également en `OncePerRequestFilter`).
2. Stockez les requêtes par adresse IP dans une `Map<String, Deque<Long>>`.
3. Limitez à 5 requêtes par minute.
4. Si le seuil est dépassé, renvoyez `429 Too Many Requests`.

Indice technique :

```
Deque<Long> deque = requests.computeIfAbsent(ip, k -> new ArrayDeque<>());
long now = System.currentTimeMillis();
while (!deque.isEmpty() && deque.peekFirst() < now - 60_000) {
    deque.pollFirst();
}
if (deque.size() >= 5) {
    response.setStatus(429);
    response.getWriter().write("Too Many Requests");
    return;
```

```
}
deque.addLast(now);
```

Questions à consigner :

1. Pourquoi ce mécanisme améliore-t-il la disponibilité du service ?
2. Comment le rendre plus efficace dans une architecture distribuée (plusieurs instances de l'API) ? *(Indice : une `Map` en mémoire locale ne suffit plus dès qu'il y a plusieurs instances — quelle solution partagée entre instances pourrait remplacer cette structure ?)*

6. Exercice 5 — HTTPS et gestion centralisée des erreurs *(nouveau)*

Objectif : appliquer deux bonnes pratiques transversales vues au Chapitre 7, absentes de la version précédente de ce TP.

6.1 Activer HTTPS en développement

1. Générez un certificat auto-signé avec `keytool` (fourni avec le JDK) :

```
keytool -genkeypair -alias tpapi -keyalg RSA -keysize 2048 \
  -storetype PKCS12 -keystore keystore.p12 -validity 365
```

2. Ajoutez dans `application.properties` :

```
server.port=8443
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=votre_mot_de_passe
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tpapi
```

3. Relancez l'application et vérifiez que `https://localhost:8443/...` répond (Postman affichera un avertissement de certificat non reconnu — c'est normal avec un certificat auto-signé, à accepter manuellement pour ce TP).

Question à consigner : en quoi HTTPS protège-t-il concrètement la clé API et le JWT que vous transportez dans l'en-tête `Authorization`, vus aux Exercices 1 et 2 ?

6.2 Gestion centralisée des erreurs

Plutôt que de renvoyer des codes bruts (401, 429) sans contexte, uniformisez toutes les réponses d'erreur de l'API avec un gestionnaire global (rappel du TP2) :

```
import org.springframework.web.bind.annotation.*;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import java.time.LocalDateTime;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(org.springframework.web.bind.MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(
            org.springframework.web.bind.MethodArgumentNotValidException ex) {
        String message = ex.getBindingResult().getFieldErrors().get(0).getDefaultMessage();
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of(
            "timestamp", LocalDateTime.now(),
```

```
            "status", 400,
            "error", "Bad Request",
            "message", message
        ));
    }
}
```

7. Exercice 6 — Tests et validation globale

Objectif : vérifier la cohérence de l'ensemble des filtres.

Testez votre API avec Postman :

Scénario	Résultat attendu
Sans clé API	`401 Unauthorized`
Avec clé API valide	`200 OK`
Avec JWT utilisateur, sur `/utilisateurs/user`	Accès autorisé
Avec JWT utilisateur, sur `/utilisateurs/admin`	Accès refusé (`403 Forbidden`)
Avec JWT admin, sur `/utilisateurs/admin`	Accès autorisé
Plus de 5 requêtes en moins d'une minute	`429 Too Many Requests`
Description contenant `<script>`	Le script est neutralisé dans la donnée stockée
Requête en `http://` plutôt que `https://` (après l'Exercice 5)	Échec de connexion ou redirection, selon votre configuration

Questions de synthèse à consigner :

1. Quelle est la différence conceptuelle entre authentification et autorisation ?
2. Quel rôle joue le chiffrement dans le protocole HTTPS ?
3. Pourquoi est-il essentiel de combiner plusieurs mécanismes (authentification + autorisation + filtrage + rate limiting + HTTPS) plutôt que de se reposer sur un seul ?

8. Pour aller plus loin (facultatif)

- Ajoutez une vérification de l'origine des requêtes (CORS) via `WebMvcConfigurer`.
- Remplacez les identifiants simulés par une base de données d'utilisateurs, avec mots de passe hachés (`BCryptPasswordEncoder`).
- Remplacez le `RateLimitFilter` maison par la bibliothèque **Bucket4j**, plus robuste et adaptée à la production.
- Documentez vos endpoints avec **springdoc-openapi** (voir le Guide de Référence du TP2).

9. Rapport et soumission

Votre rapport doit inclure :

1. Le code source complet du projet.
2. Une capture d'écran Postman pour chaque ligne du tableau de l'Exercice 6.
3. Les réponses rédigées à toutes les questions de chaque exercice.
4. Une réflexion finale (10 lignes) sur la question de synthèse n°3.

10. Barème indicatif

Critère	Points
Exercice 1 — API Key	3
Exercice 2 — JWT et rôles	5
Exercice 3 — XSS et injections	3
Exercice 4 — Rate limiting	3
Exercice 5 — HTTPS et gestion des erreurs	3
Exercice 6 — Tests et questions de synthèse	2
Qualité du rapport	1
Total	**20**